

**Министерство науки и высшего образования Российской
Федерации ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«Процедуры, функции, триггеры в PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Голинский Данила Владимирович

Факультет прикладной информатики

Группа К3240

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2025

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2026/2027

Введение

В рамках данной лабораторной работы были изучены и практически применены механизмы расширения функциональности СУБД PostgreSQL с использованием процедурного языка PL/pgSQL. Разработанные объекты (хранимые функции, хранимые процедуры и триггеры) позволяют перенести часть бизнес-логики приложения непосредственно на уровень базы данных, что повышает производительность, снижает сетевой трафик между приложением и СУБД, а также обеспечивает строгую целостность и аудит финансовых и персональных данных абонентов интернет-провайдера.

Раздел 1: Хранимые функции

Хранимые функции используются для вычислений и возврата значений. Были реализованы функции трех типов: возвращающая таблицу, скалярная и статистическая.

1.1. Функция получения сводной информации об абоненте (get_subscriber_info)

Тип: RETURNS TABLE. Позволяет быстро извлечь актуальные личные данные абонента, его текущий баланс, а также наименование и дату подключения активного тарифного плана (где end_date IS NULL).

```
CREATE OR REPLACE FUNCTION get_subscriber_info(p_id INT)
RETURNS TABLE(
    subscriber_id    INT,
    full_name        VARCHAR,
    phone_number     VARCHAR,
    city             VARCHAR,
    current_balance  DECIMAL,
    tariff_name      VARCHAR,
    tariff_start     DATE
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        s.subscriber_id,
        s.full_name,
        s.phone_number,
        s.city,
        s.current_balance,
        tp.tariff_name,
        st.start_date
    FROM subscribers s
    LEFT JOIN subscriber_tariffs st
        ON s.subscriber_id = st.subscriber_id AND st.end_date IS NULL
```

```

LEFT JOIN tariff_plans tp
  ON st.tariff_id = tp.tariff_id
WHERE s.subscriber_id = p_id;
END;
$$;

```

Демонстрация и ожидаемый результат:

Query

Query History

Scratch Pad X

1 SELECT * FROM get_subscriber_info(1);

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

	subscriber_id integer	full_name character varying	phone_number character varying	city character varying	current_balance numeric
1	1	Голинский Данила Владимирович	+79111234567	Санкт-Петербург	550.0

```
SELECT * FROM get_subscriber_info(1);
```

1.2. Скалярная функция подсчета задолженности (calculate_subscriber_debt)

Тип: RETURNS DECIMAL(10,2). Функция агрегирует суммы всех платежей абонента, имеющих статус 'Просрочен'. Использует COALESCE во избежание возврата NULL при отсутствии долгов.

```

CREATE OR REPLACE FUNCTION calculate_subscriber_debt(p_id INT)
RETURNS DECIMAL(10,2)
LANGUAGE plpgsql
AS $$
DECLARE

```

```

    v_debt DECIMAL(10,2);
BEGIN
    SELECT COALESCE(SUM(amount), 0)
    INTO v_debt
    FROM payments
    WHERE subscriber_id = p_id
        AND payment_status = 'Просрочен';

    RETURN v_debt;
END;
$$;

```

Демонстрация и ожидаемый результат:

```

INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
VALUES (2, 350.00, '2025-03-01', '2025-02-15', 'Просрочен');

SELECT calculate_subscriber_debt(2) AS total_debt; -- Ожидается: 350.00
SELECT calculate_subscriber_debt(1) AS total_debt; -- Ожидается: 0.00

```

1.3. Функция ежемесячной статистики звонков (get_monthly_call_stats)

Тип: RETURNS TABLE. Принимает идентификатор абонента, целевой год и месяц, вычисляя общее количество совершенных звонков, их суммарную длительность и совокупную стоимость.

```

CREATE OR REPLACE FUNCTION get_monthly_call_stats(
    p_subscriber_id INT,
    p_year          INT,
    p_month         INT
)
RETURNS TABLE(
    total_calls    BIGINT,
    total_minutes  DECIMAL,
    total_cost     DECIMAL
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        COUNT(*)::BIGINT          AS total_calls,
        COALESCE(SUM(duration_minutes), 0) AS total_minutes,
        COALESCE(SUM(cost), 0)      AS total_cost
    FROM calls
    WHERE subscriber_id = p_subscriber_id
        AND EXTRACT(YEAR FROM call_date) = p_year
        AND EXTRACT(MONTH FROM call_date) = p_month;

```

```
END;  
$$;
```

Демонстрация и ожидаемый результат:

```
SELECT * FROM get_monthly_call_stats(1, 2025, 3);
```

Раздел 2: Хранимые процедуры

В отличие от функций, процедуры предназначены для выполнения последовательности действий, изменяющих состояние БД, поддерживают управление транзакциями и вызываются оператором CALL.

2.1. Регистрация платежа и пополнение баланса (add_payment)

Процедура инкапсулирует логику добавления записи в финансовую таблицу платежей и инкрементирует баланс абонента в таблице subscribers, если статус транзакции равен 'Оплачен'. Реализована строгая валидация входных данных (проверка на положительную сумму платежа и существование абонента).

Примечание по совместимости типов данных: Во избежание ошибок несовместимости типов в различных конфигурациях баз данных, входной параметр p_status принимает значение типа VARCHAR. СУБД PostgreSQL выполняет автоматическое неявное приведение строковых данных к целевому типу столбца (в данном случае к типу public.payment_status_enum) с помощью явного кастинга.

```
CREATE OR REPLACE PROCEDURE add_payment(  
    p_subscriber_id INT,  
    p_amount         DECIMAL(10,2),  
    p_payment_date   DATE,  
    p_due_date       DATE,  
    p_status         VARCHAR DEFAULT 'Оплачен'  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- 1. Валидация: сумма должна быть положительной  
    IF p_amount <= 0 THEN  
        RAISE EXCEPTION 'Сумма платежа должна быть больше нуля. Получено: %', p_amount;  
    END IF;  
  
    -- 2. Валидация: абонент должен существовать  
    IF NOT EXISTS (SELECT 1 FROM subscribers WHERE subscriber_id = p_subscriber_id) THEN  
        RAISE EXCEPTION 'Абонент с ID % не найден.', p_subscriber_id;
```

```
END IF;

-- 3. Регистрация платежа (приведение к public.payment_status_enum)
INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
VALUES (p_subscriber_id, p_amount, p_payment_date, p_due_date,
p_status::public.payment_status_enum);

-- 4. Обновление баланса абонента (только для оплаченных платежей)
IF p_status = 'Оплачен' THEN
    UPDATE subscribers
    SET current_balance = current_balance + p_amount
    WHERE subscriber_id = p_subscriber_id;
END IF;

RAISE NOTICE 'Платёж на сумму % для абонента % успешно зарегистрирован.', p_amount,
p_subscriber_id;
END;
$$;
```

Демонстрация, проверки и ожидаемый результат:

Query Query History

```
1 SELECT current_balance FROM subscribers WHERE sub
2 CALL add_payment(1, 500.00, CURRENT_DATE, CURRENT
3 SELECT current_balance FROM subscribers WHERE sub
4
```

Data Output Messages Notifications



	current_balance numeric (10,2)
1	1050.00

```
-- 1. Проверка успешного сценария платежа
SELECT current_balance FROM subscribers WHERE subscriber_id = 1;
CALL add_payment(1, 500.00, CURRENT_DATE, CURRENT_DATE, 'Оплачен');
SELECT current_balance FROM subscribers WHERE subscriber_id = 1;

-- 2. Проверка валидации на отрицательную сумму (Вызовет EXCEPTION)
CALL add_payment(1, -50.00, CURRENT_DATE, CURRENT_DATE, 'Оплачен');

-- 3. Проверка валидации на несуществующего абонента (Вызовет EXCEPTION)
CALL add_payment(9999, 100.00, CURRENT_DATE, CURRENT_DATE, 'Оплачен');
```

2.2. Смена тарифного плана абонента (change_subscriber_tariff)

Обеспечивает историчность тарификации. Процедура закрывает текущий активный тариф абонента, выставляя `end_date = CURRENT_DATE`, и мгновенно открывает запись с новым тарифным планом.

```
CREATE OR REPLACE PROCEDURE change_subscriber_tariff(  
    p_subscriber_id INT,  
    p_new_tariff_id INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Закрываем текущий активный тариф  
    UPDATE subscriber_tariffs  
    SET end_date = CURRENT_DATE  
    WHERE subscriber_id = p_subscriber_id  
        AND end_date IS NULL;  
  
    -- Подключаем новый тариф  
    INSERT INTO subscriber_tariffs (subscriber_id, tariff_id, start_date)  
    VALUES (p_subscriber_id, p_new_tariff_id, CURRENT_DATE);  
  
    RAISE NOTICE 'Абонент % переключён на тариф ID %.', p_subscriber_id, p_new_tariff_id;  
END;  
$$;
```

Демонстрация, проверки и ожидаемый результат:

```
1 SELECT * FROM subscriber_tariffs WHERE subscriber_id = 1;
2 CALL change_subscriber_tariff(1, 2);
3 SELECT * FROM subscriber_tariffs WHERE subscriber_id = 1;
4
5
```

	subscriber_id [PK] integer	tariff_id [PK] integer	start_date [PK] date	end_date date	created_at timestamp without time zone
1	1	2	2026-06-21	[null]	2026-06-21 16:36:56.102924
2	1	3	2025-01-10	2026-06-21	2026-05-09 19:26:14.287895


```
SELECT * FROM subscriber_tariffs WHERE subscriber_id = 1 AND end_date IS NULL;
CALL change_subscriber_tariff(1, 2);
SELECT * FROM subscriber_tariffs WHERE subscriber_id = 1 ORDER BY start_date DESC;
```

2.3. Полная деактивация абонента (deactivate_subscriber)

Используется при расторжении договора. Процедура закрывает все открытые тарифы и дополнительные услуги абонента текущей датой. Внедрен механизм получения количества измененных строк через GET DIAGNOSTICS.

```
CREATE OR REPLACE PROCEDURE deactivate_subscriber(p_subscriber_id INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    -- Закрываем все активные тарифы
    UPDATE subscriber_tariffs SET end_date = CURRENT_DATE WHERE subscriber_id =
p_subscriber_id AND end_date IS NULL;
    GET DIAGNOSTICS v_count = ROW_COUNT;
    RAISE NOTICE 'Закрыто тарифов: %', v_count;

    -- Закрываем все активные дополнительные услуги
    UPDATE subscriber_services SET deactivation_date = CURRENT_DATE WHERE subscriber_id =
p_subscriber_id AND deactivation_date IS NULL;
    GET DIAGNOSTICS v_count = ROW_COUNT;
    RAISE NOTICE 'Деактивировано доп. услуг: %', v_count;
END;
$$;
```

Демонстрация, проверки и ожидаемый результат:

Query

Query History

Scratch Pad

1

CALL deactivate_subscriber(1);

2

SELECT * FROM subscriber_tariffs WHERE subscriber_id = 1;

3

SELECT * FROM subscriber_services WHERE subscriber_id = 1;

4

Data Output

Messages

Notifications

	subscriber_id [PK] integer	service_id [PK] integer	activation_date [PK] date	deactivation_date date
1	1	1	2025-01-12	2026-06-21

```
CALL deactivate_subscriber(1);
SELECT * FROM subscriber_tariffs WHERE subscriber_id = 1;
SELECT * FROM subscriber_services WHERE subscriber_id = 1;
```

Раздел 3: Триггеры и триггерные функции

Триггеры автоматизируют контроль за бизнес-логикой и целостностью данных непосредственно при выполнении DML-операций. В систему добавлены новые триггеры для контроля за дебиторской задолженностью и автоустановки статусов просрочки.

3.1. Предупреждение о негативном балансе (trg_check_balance_before_payment)

Тип: BEFORE INSERT ON payments (FOR EACH ROW). Проверяет состояние счета абонента перед регистрацией платежа. При отрицательном балансе СУБД генерирует предупреждение (WARNING), не блокируя вставку.

```
CREATE OR REPLACE FUNCTION fn_check_balance_before_payment()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_balance DECIMAL(10,2);
```

```

BEGIN
    SELECT current_balance INTO v_balance FROM subscribers WHERE subscriber_id =
NEW.subscriber_id;
    IF v_balance < 0 THEN
        RAISE WARNING 'Абонент % имеет отрицательный баланс: %', NEW.subscriber_id,
v_balance;
    END IF;
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_check_balance_before_payment
BEFORE INSERT ON payments FOR EACH ROW EXECUTE FUNCTION fn_check_balance_before_payment();

```

Демонстрация, проверки и ожидаемый результат:

Query	Query History	Scra
1	UPDATE subscribers SET current_balance = -150.00	
2	INSERT INTO payments (subscriber_id, amount, payr	
3	VALUES (2, 200.00, CURRENT_DATE, CURRENT_DATE, 'O	
4		

Data Output	Messages	Notifications
WARNING: Абонент 2 имеет отрицательный баланс: -150.00		
INSERT 0 1		
Query returned successfully in 68 msec.		

```

UPDATE subscribers SET current_balance = -150.00 WHERE subscriber_id = 2;
INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
VALUES (2, 200.00, CURRENT_DATE, CURRENT_DATE, 'Ожидает');

```

3.2. Логирование и аудит изменений тарифов (trg_log_tariff_change)

Тип: AFTER INSERT ON subscriber_tariffs. Реализует паттерн «След аудита» (Audit Trail). При назначении любого нового тарифа триггер автоматически создает запись в специализированной таблице логов tariff_change_log.

```

CREATE TABLE IF NOT EXISTS tariff_change_log (
    log_id          SERIAL PRIMARY KEY,
    subscriber_id INT NOT NULL,
    tariff_id       INT NOT NULL,
    changed_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action          VARCHAR(50) DEFAULT 'НОВЫЙ ТАРИФ'
);

CREATE OR REPLACE FUNCTION fn_log_tariff_change()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO tariff_change_log (subscriber_id, tariff_id, action)
    VALUES (NEW.subscriber_id, NEW.tariff_id, 'НОВЫЙ ТАРИФ');
    RAISE NOTICE 'Аудит: абонент % подключил тариф % в %', NEW.subscriber_id,
NEW.tariff_id, NOW();
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_log_tariff_change
AFTER INSERT ON subscriber_tariffs FOR EACH ROW EXECUTE FUNCTION fn_log_tariff_change();

```

Демонстрация, проверки и ожидаемый результат:

QueryQuery HistoryScratch Pad X

```

1 INSERT INTO subscriber_tariffs (subscriber_id, tariff_id, start_date, end_date)
2 VALUES (2, 3, CURRENT_DATE, NULL);
3 SELECT * FROM tariff_change_log;
4

```

Data OutputMessagesNotifications

	log_id [PK] integer	subscriber_id integer	tariff_id integer	changed_at timestamp without time zone	action character varying (50)
1	1	2	3	2026-06-21 16:39:56.682967	НОВЫЙ ТАРИФ

```

INSERT INTO subscriber_tariffs (subscriber_id, tariff_id, start_date, end_date)
VALUES (2, 3, CURRENT_DATE, NULL);
SELECT * FROM tariff_change_log;

```

3.3. Предотвращение дублирования активных тарифов (trg_prevent_duplicate_active_tariff)

Тип: BEFORE INSERT ON subscriber_tariffs. Критически важный триггер целостности. Запрещает добавлять абоненту тариф, если у него уже имеется запись с незаполненной датой окончания (end_date IS NULL), полностью блокируя некорректную транзакцию через RAISE EXCEPTION.

```
CREATE OR REPLACE FUNCTION fn_prevent_duplicate_active_tariff()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count FROM subscriber_tariffs WHERE subscriber_id =
NEW.subscriber_id AND end_date IS NULL;
    IF v_count > 0 THEN
        RAISE EXCEPTION 'Абонент % уже имеет активный тариф. Сначала закройте текущий
тариф.', NEW.subscriber_id;
    END IF;
    RETURN NEW;
END; $$;

CREATE OR REPLACE TRIGGER trg_prevent_duplicate_active_tariff
BEFORE INSERT ON subscriber_tariffs FOR EACH ROW EXECUTE FUNCTION
fn_prevent_duplicate_active_tariff();
```

Демонстрация, проверки и ожидаемый результат:

The screenshot shows a database client interface with a 'Query' tab selected. The query text is as follows:

```
1 SELECT * FROM subscriber_tariffs WHERE subscriber_id = 2
2 -- Следующая строка вызовет ошибку:
3 INSERT INTO subscriber_tariffs (subscriber_id, tariff_id, end_date)
4 VALUES (2, 1, CURRENT_DATE, NULL);
5
```

Below the query editor, the 'Messages' tab is active, displaying an error message:

```
ERROR: Абонент 2 уже имеет активный тариф. Сначала закройте текущий тариф.
CONTEXT: PL/pgSQL function fn_prevent_duplicate_active_tariff() line 6 at RAISE
```

At the bottom, the 'SQL state' is shown as P0001.

```

SELECT * FROM subscriber_tariffs WHERE subscriber_id = 2 AND end_date IS NULL;
-- Следующая строка вызовет ошибку:
INSERT INTO subscriber_tariffs (subscriber_id, tariff_id, start_date, end_date)
VALUES (2, 1, CURRENT_DATE, NULL);

```

3.4. Автоматическая просрочка платежей (trg_auto_set_overdue_status)

Тип: BEFORE INSERT ON payments. Автоматически устанавливает статус 'Просрочен', если при вставке платежа due_date уже прошла, а статус не указан явно как 'Оплачен'. Модифицирует данные ДО записи — меняет NEW.payment_status. Это освобождает клиентское приложение от необходимости самому следить за датами: СУБД выполнит проверку на системном уровне.

```

CREATE OR REPLACE FUNCTION fn_auto_set_overdue_status()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Если срок оплаты уже прошёл И платёж НЕ помечен как оплаченный
    IF NEW.due_date < CURRENT_DATE AND NEW.payment_status <>
'Оплачен'::public.payment_status_enum THEN
        NEW.payment_status := 'Просрочен'::public.payment_status_enum;
        RAISE NOTICE 'Платёж для абонента % автоматически помечен как Просрочен
(срок: %)',
            NEW.subscriber_id, NEW.due_date;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_auto_set_overdue_status
BEFORE INSERT ON payments
FOR EACH ROW
EXECUTE FUNCTION fn_auto_set_overdue_status();

```

Демонстрация, проверки и ожидаемый результат:

Query
Query History
Scratch Pad X

```

1  -- Вставляем платёж с просроченной датой и статусом
2  INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
3  VALUES (1, 600.00, '2025-01-01', '2025-01-01', 'Ожидает');
4
5  -- Смотрим результат: статус должен автоматически измениться на 'Просрочен'
6  SELECT payment_id, subscriber_id, amount, due_date, payment_status
7  FROM payments
8  ORDER BY payment_id DESC LIMIT 1;
9  -- Ожидаемый результат: payment_status = 'Просрочен'
10

```

Data Output
Messages
Notifications

	payment_id [PK] integer	subscriber_id integer	amount numeric (10,2)	due_date date	payment_status public.payment_status_enum
1	7	1	600.00	2025-01-01	Просрочен

```

-- Вставляем платёж с просроченной датой и статусом 'Ожидает':
INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
VALUES (1, 600.00, '2025-01-01', '2025-01-01', 'Ожидает':public.payment_status_enum);

-- Смотрим результат: статус должен автоматически измениться на 'Просрочен'
SELECT payment_id, subscriber_id, amount, due_date, payment_status
FROM payments
ORDER BY payment_id DESC LIMIT 1;
-- Ожидаемый результат: payment_status = 'Просрочен'

-- Оплаченный платёж НЕ меняется триггером:
INSERT INTO payments (subscriber_id, amount, payment_date, due_date, payment_status)
VALUES (1, 200.00, '2025-01-01', '2025-01-01', 'Оплачен':public.payment_status_enum);

SELECT payment_id, subscriber_id, amount, due_date, payment_status
FROM payments
ORDER BY payment_id DESC LIMIT 1;
-- Ожидаемый результат: payment_status = 'Оплачен'

```

3.5. Блокировка удаления абонентов-должников (trg_block_delete_indebted_subscriber)

Тип: BEFORE DELETE ON subscribers. Блокирует удаление абонента, если у него есть хотя бы один платёж со статусом 'Просрочен' или 'Ожидает'. Защищает бизнес-данные: нельзя случайно утерять данные о должнике. Если долгов нет — удаление проходит в обычном режиме.

```
CREATE OR REPLACE FUNCTION fn_block_delete_indebted_subscriber()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_debt_count INT;
    v_debt_sum    DECIMAL(10,2);
BEGIN
    SELECT COUNT(*), COALESCE(SUM(amount), 0)
    INTO v_debt_count, v_debt_sum
    FROM payments
    WHERE subscriber_id = OLD.subscriber_id
        AND payment_status IN ('Просрочен'::public.payment_status_enum,
    'Ожидает'::public.payment_status_enum);

    IF v_debt_count > 0 THEN
        RAISE EXCEPTION
            'Нельзя удалить абонента "%" (ID=%). Незакрытых платежей: % на сумму % руб.
Сначала закройте все задолженности.',
            OLD.full_name, OLD.subscriber_id, v_debt_count, v_debt_sum;
    END IF;

    -- Долгов нет — разрешаем удаление
    RAISE NOTICE 'Абонент "%" (ID=%) удаляется. Задолженностей не найдено.',
    OLD.full_name, OLD.subscriber_id;
    RETURN OLD;
END;
$$;

CREATE OR REPLACE TRIGGER trg_block_delete_indebted_subscriber
BEFORE DELETE ON subscribers
FOR EACH ROW
EXECUTE FUNCTION fn_block_delete_indebted_subscriber();
```

Демонстрация, проверки и ожидаемый результат:

Query	Query History	Scratch
1	-- Убеждаемся, что у абонента 2 есть просроченный	
2	SELECT subscriber_id, payment_status, amount	
3	FROM payments	
4	WHERE subscriber_id = 2 AND payment_status IN ('Просрочен', 'Ожидает')	
5		
6	-- Попытка удалить абонента с долгом (вызовет ошибку)	
7	-- DELETE FROM subscribers WHERE subscriber_id = 2;	
8		
9	-- Правильный сценарий: закрываем долги, затем выполняем удаление	
10	UPDATE payments	
11	SET payment_status = 'Оплачен'::public.payment_status_enum	
12	WHERE subscriber_id = 2 AND payment_status IN ('Просрочен', 'Ожидает');	
13		
14	-- Теперь удаление пройдет успешно:	
15	-- DELETE FROM subscribers WHERE subscriber_id = 2;	
16		

Data Output	Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈		
subscriber_id	payment_status	amount
integer	public.payment_status_enum	numeric (10,2)

```
-- Убеждаемся, что у абонента 2 есть просроченный платёж:
SELECT subscriber_id, payment_status, amount
FROM payments
WHERE subscriber_id = 2 AND payment_status IN ('Просрочен'::public.payment_status_enum,
'Ожидает'::public.payment_status_enum);

-- Попытка удалить абонента с долгом (вызовет ошибку):
-- DELETE FROM subscribers WHERE subscriber_id = 2;

-- Правильный сценарий: закрываем долги, затем выполняем удаление
UPDATE payments
SET payment_status = 'Оплачен'::public.payment_status_enum
WHERE subscriber_id = 2 AND payment_status IN ('Просрочен'::public.payment_status_enum,
'Ожидает'::public.payment_status_enum);

-- Теперь удаление пройдет успешно:
-- DELETE FROM subscribers WHERE subscriber_id = 2;
```

Раздел 4: Итоговая проверка всех объектов БД

В рамках финальной верификации целостности разработанной базы данных используются системные представления метаданных информационной схемы (information_schema). Это позволяет убедиться, что все созданные функции, процедуры и триггеры зарегистрированы СУБД и готовы к эксплуатации.

4.1. Скрипт проверки функций, процедур и триггеров в схеме

```
-- Список всех созданных функций и процедур:
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'provider_schema'
ORDER BY routine_type, routine_name;

-- Список всех триггеров:
SELECT trigger_name, event_manipulation, event_object_table, action_timing
FROM information_schema.triggers
WHERE trigger_schema = 'provider_schema'
ORDER BY event_object_table, trigger_name;

-- Общее состояние БД после всех операций:
SELECT subscriber_id, full_name, current_balance FROM subscribers;
SELECT * FROM tariff_change_log ORDER BY log_id;
```

Query

Query History

Scratch Pad X

```

1  -- Список всех созданных функций и процедур:
2  SELECT routine_name, routine_type
3  FROM information_schema.routines
4  WHERE routine_schema = 'provider_schema'
5  ORDER BY routine_type, routine_name;
6
7  -- Список всех триггеров:
8  SELECT trigger_name, event_manipulation, event_object_table
9  FROM information_schema.triggers
10 WHERE trigger_schema = 'provider_schema'
11 ORDER BY event_object_table, trigger_name;
12
13 -- Общее состояние БД после всех операций:
14 SELECT subscriber_id, full_name, current_balance
15 SELECT * FROM tariff_change_log ORDER BY log_id;
16

```

Data Output

Messages

Notifications

+

📄

▼

🗑️

🔄

⬇️

📶

	log_id [PK] integer	subscriber_id integer	tariff_id integer	changed_at timestamp without time zone	action character varying (50)
1	1	2	3	2026-06-21 16:39:56.682967	НОВЫЙ ТАРИФ

Заключение

В ходе выполнения лабораторной работы были успешно освоены ключевые концепции процедурного программирования в среде PostgreSQL. Созданная архитектура функций, процедур и триггеров позволила полностью автоматизировать финансовые операции (прием платежей, расчет долгов), гарантировать поддержку истории тарификации абонентов провайдера и исключить логические ошибки (такие как одновременное сосуществование двух активных тарифов, удаление должников или ручной контроль просрочки платежей). Все разработанные скрипты успешно прошли верификацию через системные каталоги information_schema и готовы к беспрепятственной интеграции в backend корпоративных информационных систем.